# Google Summer of Code 2025

[libssh]

## **Project:** Support for FIDO/U2F keys on the client side

### GENERAL DETAILS

**Name:** Praneeth Sarode

**Time Zone:** India (UTC+5.30)

**University:** Indian Institute of Technology, Roorkee (IIT Roorkee)

**Expected Graduation Date:** May 2027 (Currently, a Sophomore)

**Primary Email:** praneethsarode@gmail.com

**Secondary Email:** praneeth_s@cs.iitr.ac.in

**Matrix ID:** @praneethsarode:matrix.org

**GitHub:** DarkPhoenix42

**GitLab:** praneethsarode

## DESCRIPTION

**FIDO** (Fast Identity Online) security keys, or **U2F** (Universal 2nd Factor) keys, are hardware-based cryptographic devices that offer strong two-factor authentication. They provide a significantly more secure alternative to traditional password-based authentication and one-time codes and protect against a wide range of attacks, including phishing, that might compromise software-based credentials.

This project aims to enhance libssh by introducing client-side support for modern U2F/FIDO security keys. While libssh currently supports the server-side verification of these keys, this project will focus on enabling libssh clients to authenticate with SSH servers using U2F/FIDO hardware. This will bring a more secure and widely adopted authentication method to libssh clients, aligning its capabilities with leading SSH implementations like OpenSSH and enhancing its overall utility.

The **deliverables** for this project include:

1. Support for arbitrary security key middleware to be used (they must satisfy a defined API which will also be provided).
2. An internal implementation of the middleware in the case of USB/HID FIDO keys.
3. Code paths on the client side to enroll FIDO devices and create both ecdsa/ed25519 signatures.
4. Support for the certificate variants of these keys.
5. Support for webauthn ecdsa signatures.
6. Support to store attestation information returned at the time of FIDO key enrollment.
7. Updated version of keygen2.c that supports enrolling new keys on FIDO devices.
8. Updated examples and documentation that reflect the newly added functionalities.

I firmly believe that code testing is a crucial part of the development workflow. Hence, I will ensure that extensive tests are written for any code that is added so that we can be confident in the validity of the written code.

## Possible Mentor(s)

1. Jakub Jelen (*jjelen*)

## What city and country will you reside in during the summer?

For most of the summer, I will reside in **Roorkee, Uttarakhand, India,** and for a while in **Hyderabad, India**. But regardless of where I reside, I will always have access to fast internet.

## Will the proposed work address an open issue at the libssh GitLab? If yes, paste the link to the issue.

The proposed work will address this issue on gitlab. Although some work has been done towards this issue in this MR, It is incomplete and needs major additions to work. It also does not have tests and proper documentation.

## What benefits does your proposed work have for libssh and its community?

My proposed work to add FIDO key support on the client side offers the following key benefits for libssh and its community:

1. **Enhanced Security:** FIDO/U2F security keys provide hardware-backed two-factor authentication, which is significantly more secure compared to traditional password-based authentication or even software-based one-time codes

2. **Expanded Authentication Options:** Adding FIDO/U2F support brings libssh in line with OpenSSH, making it a more competitive choice for modern authentication needs and broadening its adoption in security-focused applications.
3. **Improved Usability for Developers and Enterprises:** Security-conscious organizations, cloud platforms, and DevOps tools will benefit from seamless integration with FIDO/U2F keys, simplifying compliance with security best practices and industry standards.

This project will make libssh a more secure and future-proof SSH library, improving authentication standards while ensuring long-term sustainability and usability for developers and organizations.

## Why are you the right person to work on this project?

I am enthusiastic about contributing to this project due to my interests in cryptography and protocol design. I have always been fascinated by the design of protocols, and how they're able to offer support for so many features and the possibility to extend it even further while ensuring backward compatibility. SSH is one such protocol that I have used very frequently in my life. When I learned that libssh was participating in GSoC with a project related to FIDO keys, I was excited about the opportunity to combine my passions for cryptography and protocol design and implementation.

To prepare for this project, I have dedicated considerable time to studying OpenSSH's implementation of FIDO key support and reviewing the documentation for libfido2. This has provided me with a solid understanding of the steps required to implement FIDO/U2F key support in libssh. I have also spent a significant amount of time on contributing meaningful code to the libssh codebase, so that I am familiar with both the codebase and the development workflow.

Also, participating in CTFs (Capture The Flag events) with InfoSecIITR , currently ranked 12th in the world on ctftime.org, as a cryptographer and reverse engineer has given me a good grasp on Elliptic Curve Cryptography, and debugging binaries and how libraries work.

I am eager to learn and experiment with new technologies, and my prior involvement in various teams and open-source projects has equipped me with knowledge of the software development process, best practices, and recommendations. Hence, I am confident in my ability to meet the high standards expected by open-source organizations. Collaborating with experienced professionals as mentors will be an invaluable learning experience for me.

## Is your proposal for a medium (175h) or big (350h) project?

My proposal is for a **350-hour** (big) project, as it involves not only adding support for FIDO/U2F keys in libssh but also ensuring comprehensive integration. This includes writing extensive test cases to verify functionality, updating relevant documentation, and updating the examples to include the new feature. Given the complexity of implementing secure authentication mechanisms and ensuring compatibility with existing workflows, a larger time commitment is necessary to deliver a well-tested and fully documented solution.

## How do you plan to achieve completion of your project?

- The U2F protocol cannot be trivially used as an SSH protocol key type as both the inputs to the signature operation and the resultant signature differ from those specified for SSH. For similar reasons, integration of U2F devices cannot be achieved via the PKCS#11 API.

- Also, U2F devices can communicate over various protocols such as NFC, USB/HID. Here, we can take inspiration for openSSH, and declare an API, and any middleware that satisfies that API can be dynamically loaded

and used to perform operations such as enrolling new keys, signing data, and listing all connected keys, so that support for a new device using a new protocol can be added just by providing a middleware library for it, instead of having to tinker with the libssh codebase.

- We should also provide an internal implementation of the middleware for the common case of USB/HID FIDO devices using libfido2 as openSSH does.

**The following are the steps that I would take to achieve the objective:**

- I would create a new file that contains helper functions to load the middleware library and interact with it using functions such as dlopen, and check whether the middleware satisfies the API using dlsym and write unit tests for this file utilising sk-dummy.so as the middleware library.

- I would then work on the internal implementation for the USB-HID using libfido2. And test the middleware loading functions using this implementation too.

- My focus would then be to add code paths to the pki.c file, or create a separate file which contains helper functions to serialize data into the needed format for fido key signature, and then deserialize the resulting signature as per the FIDO specification and properly handle flags such as USER_PRESENCE_REQUIRED and USER_VERIFICATION_REQUIRED. This includes signatures for both the ecdsa-sha2 and ed25519 case.

- I would then write some integration tests to check against an openssh server whether the added code paths work properly utilising sk-dummy.so as the middleware.

- The sk-dummy.so is designed to return valid responses that the actual hardware would return. So, we can load it as the middleware and use the functions it provides to simulate fido keys without the need for actual hardware. This is a really important step as it allows us to add tests to the CI pipeline where hardware support is not available.

- Then, I plan on introducing support for the certificate variants of the keys by adding code for proper serialisation of the additional information in certificates, and again write integration tests for these.

- Then, I would work on adding support for webauthn signatures, and similarly write tests for these.

- I would then work on adding functions to store attestation information returned at the time of key enrollment as a file, and similarly write tests for these.

- Then, I would start working on providing examples and documentation for the added code, by first extending keygen2.c to support key generation using fido keys.

- I would then edit the existing documentation to add an example of using fido key for authentication and if needed, create a new section of documentation for this.

**Please provide a sequence of tasks and subtasks and how long (days) you estimate it will take you to complete each of them. Highlight important milestones/deliverables. (Suggestion: create a table covering the whole period).**

The deliverables until [4] as listed in the DESCRIPTION section are planned to be delivered before the mid evaluation and the rest after the mid evaluation.

| Pre GSOC Period | |
|---|---|
| April 8 - May 4 | • I would try completing the remaining issues that I am working on in libssh as well as continue research on the way openssh has handled fido keys and reading the libfido2 documentation. |
| **Community Bonding Period** | |
| May 15 - May 28 | • With the help of my mentor(s), I'll decide on the API for the middleware library and get an idea of the modifications needed to existing libssh structures and enums to add support for the new key and certificate variants and identify more relevant parts of the codebase.<br>• I would talk with my mentor(s) and get a better idea of each subtask and modify my timeline accordingly.<br>• I would discuss with my mentor(s) regarding what functions should be added to the libssh API and what functions should remain internal. |
| **Coding Period** | |
| Week 1<br>(June 2 - June 9) | • Update the CMake files to find libfido2, add option to build with or without FIDO support and add sk-dummy.so to CI testing images here.<br>• Implement the decided upon API as a header file and create the required structs.<br>• Start working on the file which loads any specified middleware and validates that it satisfies the required API. |
| Week 2<br>(June 10 - June 16) | • Add unit tests for the middleware loading file using sk-dummy.so |

| | |
|---|---|
| | • Start working on the internal implementation of the middleware for USB/HID case using libfido2 |
| Week 3<br>(June 17 - June 23) | • Continue working on the internal implementation. |
| Week 4<br>June 24 - June 30 | • Extend the previous tests for middleware loading, to include the internal implementation.<br>• Verify that the internal implementation works, using an actual fido key.<br>• Start working on adding code to serialise data in order to prepare it for signing and deserialise the signatures for both ecdsa-sha2 and ed25519. |
| Week 5<br>(July 01 - July 07) | • Continue working on the ecdsa-sha2 and ed25519 signatures |
| Week 6<br>(July 08 - July 14) | • Start writing integration tests against an openSSH server to ensure correct signing. |
| Week 7<br>(July 15 - July 21) | • Start working on support for the certificate variants of these keys.<br>• Buffer time to complete pending tasks, if any |
| **Mid evaluation** | |
| Week 8<br>(July 29 - Aug 04) | • Finish adding support for certificates.<br>• Start working on adding integration tests for the certificates against openSSH |
| Week 10<br>(Aug 05 - Aug 11) | • Add support for webauthn signatures<br>• Start writing integration tests against an openSSH server for webauthn signatures. |
| Week 11<br>(Aug 12 - Aug 18) | • Work on functions to support storing attestation information.<br>• Start working on keygen2.c to extend its |

| | |
|---|---|
| | functionality and incorporate fido key generation support. |
| Week 12<br>(Aug 19 - Aug 25) | • Update examples such as ssh_client.c to reflect the fido key changes. |
| Week 13<br>(Aug 26 - Sept 01) | • Add documentation related to fido key support, and prepare for final evaluation. |
| **Final evaluation** | |

## What are your past experiences with the open source world as a user and as a contributor?

As a dedicated member of the open-source community, I have engaged both as a user and a contributor, gaining valuable insights and experience.

**As a User:**

My journey began with utilizing many open-source tools and libraries to support my academic and personal projects such as Git, Docker, VS Code, Wireshark, pwndbg, ..etc. This exposure allowed me to appreciate the collaborative efforts behind these projects and understand their practical applications.

**As a Contributor:**

When I joined IIT Roorkee, I became an active member of SDSLabs, a student-driven technical group and have contributed to various open-source projects. SDSLabs focuses on fostering technological innovation and has developed numerous projects available on GitHub.

These experiences have enhanced my technical skills and reinforced my commitment to the open-source ethos of collaboration and knowledge sharing.

## Previous contributions to libssh:

I began contributing to libssh around February, 2025. The following are all of my contributions to libssh:

| Merge Request | Description | Status |
|---|---|---|
| https://gitlab.com/libssh/libssh-mirror/-/merge_requests/566 | Adds support the sftp-users-groups-by-id @openssh.com extension | Merged |
| https://gitlab.com/libssh/libssh-mirror/-/merge_requests/585 | Add support for mbedTLS implementation of Curve25519 for ECDH key exchange | Merged |
| https://gitlab.com/libssh/libssh-mirror/-/merge_requests/587 | Add a callback for direct-tcpip requests and write unit tests to verify that the callback is called appropriately | Merged |
| https://gitlab.com/libssh/libssh-mirror/-/merge_requests/593 | Add unit tests for verify that forwarded-tcpip callback is called appropriately | Merged |
| https://gitlab.com/libssh/libssh-mirror/-/merge_requests/592 | Add support for libgcrypt's implementation of Curve25519 for ECDH key exchange | Merged |
| https://gitlab.com/libssh/libssh-mirror/-/merge_requests/598 | Fix the version detection of mbedTLS in cmake | Merged |
| https://gitlab.com/libssh/libssh-mirror/-/merge_requests/570 | Add a section on ABI versioning and symbol management to CONTRIBUTING.md | Merged |
| https://gitlab.com/libssh/ | Fix and add the | Merged |

| | | |
|---|---|---|
| libssh-mirror/-/merge_requests/591 | torture_server_x11 test to unit tests | |

I plan to continue to work on fixing some minor issues such as:

- some torture tests failing on FreeBSD,
- the client-side callbacks being called only when the WITH_SERVER macro is defined,
- updating mbedTLS curve25519 code to remove redundant copying of the private key, and also
- updating the torture_proxycommand so that it works on shells such as dash and fish.. etc.

If time permits I also plan to work on a major issue such as this.

## If available, please include links to any code you wrote for other open source projects.

**Beast:**

Beast is a service that runs on your host(maybe a bare metal server or a cloud instance) and helps manage deployment, lifetcycle, and health check of CTF challenges. It can also be used to host Jeopardy-style CTF competitions.

My PR involves migrating beast from sqlite to postgres:
https://github.com/sdslabs/beast/pull/413

**Watchdog:**

Watchdog is a personalised server access management tool (and a Slack bot) that keeps track of all the administrative rights attempts (like sudo and su) on the server (via SSH) and allows/disallows log-in attempts based on the public key of the user and logs all activity in form of slack message. It provides easy granting/revoking of access to servers to team members through pull requests on a keyhouse repository.

My PR involves a bug fix regarding faulty base64 decoding:
https://github.com/sdslabs/watchdog/pull/23

I've also worked on adding monitoring and real time notifications if anything goes wrong for our cloud infrastructure using node-exporter, Prometheus, Grafana, cAdvisor and Alert Manager.

## What other relevant projects have you worked on previously and what knowledge have you gained from working on them?

I have worked on several projects that have enhanced my skills in systems programming, security, and open-source development, directly aligning with the objectives of my proposed GSoC project. Here are some of them in order of their relevance.

**p-torrent:** Developing p-torrent, a BitTorrent client, has enhanced my understanding of network protocols and concurrent programming. These skills would be beneficial when working with a lot of deserialisation and serialisation and network communication during implementing my GSoC project.

**xv6 Operating System Labs:** Working on solutions for the labs of MIT's xv6 has provided me with insights into operating system internals and system-level programming. This experience is valuable for navigating and modifying complex codebases.

**CryptoHack Challenges:** I am currently ranked **83rd** in the world out of 75,000+ users on cryptohack.org. Solving challenges related to Isogenies and ECC has deepened my understanding of . This knowledge is directly applicable to integrating cryptographic authentication methods, such as FIDO/U2F, into libssh as they require a good understanding of ECC.

**pwn.college:** Solving the pwning challenges at pwn college has enhanced my skills in system security, exploitation techniques, and low-level programming.

**Chip8 Emulator:** Building a Chip8 Emulator has improved my proficiency in low-level programming and debugging.

**AtomicBM:** I used google's testing and benchmarking framework to benchmark the performance of std::atomic in comparison to std::mutex, which helped me familiarise myself with testing and benchmarking frameworks. This will be relevant when writing unit tests and integration tests for fido keys.

Apart from this I have worked on several other projects such as a developer toolchain for a blockchain data storage company called Walrus, some parts of the famous Nand2Tetris course and a reaction diffusion simulator using OpenCL shaders and SDL2 for rendering. I have also completed 8/10 challenges and ranked 332nd in the world, in Flare-On 11 which is a popular annual reverse engineering challenge, wherein I improved my debugging skills and understanding of binaries.

## What other time commitments, such as school work, exams, research, another job, planned vacation, etc? What are the dates for these commitments and how many hours a week do these commitments take?

I can dedicate more than 40 hours a week to this project. My college's summer vacations span from 20.05.2025 to 16.07.2025, during which I will have no classes or academic commitments, allowing me to devote 40+ hours weekly. I have my end-term examinations from 05.05.2025 to 14.05.2025, during which I will be unavailable, but I will be completely free afterward.

Once classes resume, I can easily dedicate 25 hours a week. Although classes will occupy a major part of my day, GSoC will remain a very important priority. I generally work from 5 PM to 2:30 AM IST but can adjust my schedule if needed. Other than this project, I have no other commitments. I will maintain transparency with the community and keep everyone updated on my availability. I am fully committed to this project, both in time and effort. Also, this is the only project I've applied to.

## Blogging:

I have also set up a blogging page here. While reading blogs from previous GSoC contributors, I found them useful in understanding the challenges they faced and how they approached their projects. Their experiences gave me a better idea of what to expect and how to navigate the development process. Inspired by this, I decided to start my own blog to document my journey throughout GSoC. This will not only help me track my own progress but also serve as a resource for future contributors who might work on similar projects. I plan to share updates on my work, challenges encountered and insights gained from discussions with mentors and the community.

Additionally, I hope my blog encourages more people to contribute to open source by providing a transparent look into the development process. Writing about my experience will also help me reflect on my work and improve my ability to communicate technical ideas effectively.

## Involvements after GSoC

I have already learned a lot contributing to libssh. Even after the GSoC period ends, I plan on contributing to this organization by adding to my past projects and working on open issues because of my familiarity with the technical stack and the new challenges that I am continually offered in the process.

Also, I would like to continue to work on any issue that deals with cryptography or security. Having picked up many development skills, my primary focus would be to help the project and the community grow.

## Appreciation:

I deeply appreciate the libssh community for maintaining a clean, well-structured, and high-quality codebase. The consistency in the code made it easier for me to understand the architecture and follow best practices while writing my own code.

The project's CI pipeline was especially helpful—it caught many of the subtle errors and memory leaks early in the development process, which really helped me in contributing robust code.

Beyond the technical aspects, I'm grateful to the maintainers, especially @jjelen and @eshankelkar, who guided me in solving issues and also promptly reviewed my MRs and gave their valuable suggestions, which helped me learn a lot.

## REFERENCES


- https://fidoalliance.org/specs/u2f-specs-master/fido-u2f-overview.html
- https://github.com/openssh/openssh-portable/blob/master/PROTOCOL.u2f
- https://swjm.blog/the-complete-guide-to-ssh-with-fido2-security-keys-841063a04252
- https://developers.yubico.com/libfido2/Manuals/